NSDA Level-4 Python with Django Written Questions with Answers

Part-1 (Python Advanced)

1. Explain the difference between Python List, Tuple, Set, and Dictionary with examples.

Answer:

Python has four common built-in collection data types: List, Tuple, Set, and Dictionary.

List:

- Ordered collection.
- Mutable (can be changed).
- Allows duplicate values.

Example:

```
numbers = [10, 20, 30, 20]
print(numbers)
```

Output:

```
[10, 20, 30, 20]
```

Tuple:

- Ordered collection.
- Immutable (cannot be changed).
- Allows duplicate values.

Example:

```
data = (10, 20, 30)
print(data)
```

Set:

- Unordered collection.
- Mutable.
- Does not allow duplicate values.

Example:

```
items = {10, 20, 30, 20}
print(items)
```

Output:

{10,20,30}

Dictionary:

- Stores data as key-value pairs.
- Mutable.
- Keys must be unique.

Example:

```
student = {
   "name": "John",
   "age": 25
}

print(student["name"])
```

Output:

John

---

2. Explain the difference between for loop and while loop in Python.

Answer:

Both loops are used for repetition, but their working methods are different.

for loop:

- Used when the number of iterations is known.
- Works with sequences like list, tuple, string, range.

Example:

```
for i in range(5):
   print(i)
```

while loop:

- Used when the condition controls the loop.
- Runs until the condition becomes False.

Example:

```
i = 0

while i < 5:
    print(i)
    i += 1
```

Difference:

for loop	while loop
Used for fixed iteration	Used for condition-based iteration
Works with iterable objects	Works with conditions
Less chance of infinite loop	Can create infinite loop

---

3. Explain the use of break, continue, and pass statements in Python.

Answer:

break:

`break` stops the loop completely.

Example:

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

0 1 2 3 4

---

continue:

`continue` skips the current iteration and moves to the next iteration.

Example:

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output:

0 1 3 4

---

pass:

`pass` does nothing. It is used as a placeholder.

Example:

```
if True:
    pass
```

---

4. Write the syntax and explain the Python Ternary Operator with an example.

Answer:

Python ternary operator is a short way to write if-else conditions.

Syntax:

value_if_true if condition else value_if_false

Example:

```
age = 20

result = "Adult" if age >= 18 else "Child"

print(result)
```

Output:

Adult

---

5. Explain nested loops in Python with a practical example.

Answer:

A nested loop means a loop inside another loop.

Example:

```
for i in range(3):
    for j in range(2):
        print(i,j)
```

Output:

```
0 0
0 1
1 0
1 1
2 0
2 1
```

Practical use:

- Creating tables
- Working with matrices
- Pattern printing

---

6. What are default arguments in Python functions? Explain with an example.

Answer:

Default arguments are function parameters that already have a predefined value.

Example:

```
def greet(name="User"):
    print("Hello", name)

greet()
greet("Rahim")
```

Output:

Hello User
Hello Rahim

---

7. Explain *args and **kwargs with examples.

Answer:

*args:

Used to pass multiple positional arguments.

Example:

```
def total(*numbers):
    print(numbers)

total(10,20,30)
```

Output:

(10,20,30)

**kwargs:

Used to pass multiple keyword arguments.

Example:

```
def student(**data):
    print(data)

student(name="John", age=25)
```

Output:

{'name':'John','age':25}

---

8. What is the difference between a normal function and a lambda function?

Answer:

Normal Function:

- Created using `def`.
- Can contain multiple statements.
- Has a function name.

Example:

```
def add(a,b):
    return a+b
```

Lambda Function:

- Anonymous function.
- Created using `lambda`.
- Usually contains one expression.

Example:

```
add = lambda a,b: a+b
```

---

9. Explain Python decorators and their use cases.

Answer:

A decorator is a function that modifies the behavior of another function without changing its code.

Example:

```
def decorator(func):
    def wrapper():
        print("Before function")
        func()
    return wrapper
```

Uses:

- Authentication
- Logging
- Permission checking
- Performance measurement

---

10. Explain exception handling in Python. Describe try, except, else, and finally.

Answer:

Exception handling prevents program crashes.

Structure:

```
try:
    risky_code

except:
    handle_error

else:
    execute_if_no_error

finally:
    always_execute
```

try:
Contains risky code.

except:
Handles errors.

else:
Runs when no error occurs.

finally:
Always runs.

Part-2 (Python Advanced: 11–30)

11. What is the difference between `raise` and `return` in Python?

Answer:

`return` is used to send a value back from a function.

Example:

```
def add(a, b):
```

```
    return a + b

result = add(5, 10)
print(result)
```

Output:

15

`raise` is used to manually create an exception.

Example:

```
age = -1

if age < 0:
    raise ValueError("Invalid age")
```

Difference:

return	raise
Sends data from function	Creates an error
Normal program flow	Stops normal execution
Used for output	Used for exception handling

---

12. Explain file handling in Python. Describe different file modes (`r`, `w`, `a`, `x`).

Answer:

File handling allows Python programs to create, read, write, and modify files.

Python uses the `open()` function.

Syntax:

```
file = open("filename", "mode")
```

File modes:

r (Read):

- Opens file for reading.
- Default mode.

Example:

file = open("data.txt", "r")

w (Write):

- Creates a new file or overwrites existing content.

Example:

file = open("data.txt", "w")

a (Append):

- Adds data at the end of a file.

Example:

file = open("data.txt", "a")

x (Create):

- Creates a new file.
- Gives error if file already exists.

Example:

file = open("new.txt", "x")

---

13. Explain Object-Oriented Programming (OOP) concepts in Python.

Answer:

OOP is a programming approach based on objects and classes.

Main concepts:

1. Class:
A blueprint for creating objects.

2. Object:
An instance of a class.

3. Encapsulation:
Wrapping data and methods together.

4. Inheritance:
Using properties of another class.

5. Polymorphism:
Same method behaves differently.

6. Abstraction:
Showing only important details and hiding complexity.

---

14. Explain the difference between Class and Object with examples.

Answer:

Class:

A class is a blueprint that defines properties and behaviors.

Example:

```
class Student:
    name = "John"
```

Object:

An object is an instance of a class.

Example:

```
student1 = Student()
print(student1.name)
```

Difference:

Class	Object
Blueprint	Real instance
Does not occupy memory for data	Occupies memory
Defines structure	Uses structure

---

15. Explain the purpose of the `__init__()` method in Python.

Answer:

`__init__()` is a constructor method that runs automatically when an object is created.

It initializes object attributes.

Example:

```
class Student:

    def __init__(self, name):
        self.name = name

student = Student("Rahim")

print(student.name)
```

Output:

Rahim

---

16. Explain inheritance in Python. Describe different types of inheritance.

Answer:

Inheritance allows one class to use properties and methods of another class.

Example:

```
class Animal:
    def sound(self):
        print("Animal sound")


class Dog(Animal):
    pass


dog = Dog()
dog.sound()
```

Types of inheritance:

1. Single Inheritance
   One child class inherits from one parent class.
2. Multiple Inheritance
   One class inherits from multiple classes.
3. Multilevel Inheritance
   A class inherits from a child class.
4. Hierarchical Inheritance
   Multiple classes inherit from one parent class.
5. Hybrid Inheritance
   Combination of different inheritance types.

---

17. Explain polymorphism with a Python example.

Answer:

Polymorphism means the same method name can perform different actions.

Example:

```
class Cat:

    def sound(self):
        print("Meow")


class Dog:

    def sound(self):
        print("Bark")


for animal in [Cat(), Dog()]:
    animal.sound()
```

Output:

```
Meow
Bark
```

---

18. Explain encapsulation and abstraction in Python.

Answer:

Encapsulation:

Encapsulation means combining data and methods inside a class and controlling access.

Example:

```
class Account:

    def __init__(self):
        self.__balance = 1000
```

`__balance` is private.

Abstraction:

Abstraction hides internal implementation and shows only necessary information.

Example:

Using ATM:

- User sees withdrawal option.
- Internal banking process is hidden.

---

19. Explain the difference between method overloading and method overriding.

Answer:

Method Overloading:

Same method name with different parameters.

Python does not support traditional overloading directly.

Example:

```
def add(a,b,c=0):
    return a+b+c
```

Method Overriding:

Child class changes parent class method behavior.

Example:

```
class Parent:

    def show(self):
        print("Parent")


class Child(Parent):

    def show(self):
        print("Child")
```

---

20. Explain the use of `super()` function in Python.

Answer:

`super()` is used to call methods and constructors from the parent class.

Example:

```
class Parent:

    def show(self):
        print("Parent")


class Child(Parent):

    def show(self):
        super().show()
        print("Child")
```

Output:

```
Parent
Child
```

---

21. What is Django? Explain the advantages of using Django.

Answer:

Django is a high-level Python web framework used for building secure and scalable web applications.

Advantages:

- Fast development
- Built-in authentication
- ORM support
- Security features
- Admin panel
- URL routing
- Template system
- Large community support

---

22. Explain Django MVT architecture with a diagram.

Answer:

Django follows MVT (Model View Template) architecture.

Structure:

User
|
URL
|
View
|
Model
|
Database

View
|
Template
|
User Interface

Components:

Model:
Handles database operations.

View:
Handles business logic.

Template:
Handles presentation/UI.

---

23. Explain the difference between Django Project and Django App.

Answer:

Django Project:

A complete web application configuration.

Contains:

- Settings
- URLs
- Multiple apps

Django App:

A specific functionality/module inside a project.

Example:

Project:

E-commerce Website

Apps:

users
products
orders

---

24. Explain the structure of a Django project.

Answer:

Common Django project structure:

project/

```
|
├── manage.py
|
├── settings.py
├── urls.py
├── asgi.py
└── wsgi.py
```

App structure:

```
app/
|
├── models.py
├── views.py
├── admin.py
├── forms.py
└── urls.py
```

---

25. Explain the purpose of important Django files.

Answer:

settings.py

Contains project configuration:

- Database settings
- Installed apps
- Middleware
- Static files

urls.py

Handles URL routing.

views.py

Contains business logic and request handling.

models.py

Defines database structure.

admin.py

Registers models in Django admin panel.

---

26. Explain the Django request-response lifecycle.

Answer:

Django request lifecycle:

1. User sends request.
2. URL dispatcher matches URL.
3. View function/class receives request.
4. View processes data.
5. Model communicates with database.
6. Template generates response.
7. Response is returned to user.

Flow:

Request
|
URL
|
View
|
Model
|
Database
|
Template
|
Response

---

27. Explain the difference between Function-Based Views (FBV) and Class-Based Views (CBV).

Answer:

FBV:

- Written using functions.
- Simple and easy for small applications.

Example:

```
def home(request):
    return HttpResponse("Home")
```

CBV:

- Written using classes.
- Supports inheritance and reusable code.

Example:

```
class HomeView(View):
    pass
```

Difference:

FBV	CBV
Function based	Class based
Simple	More reusable
Less code	More structured

---

28. What is Django URL Routing? Explain with an example.

Answer:

URL Routing maps user requests to specific views.

Example:

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("home/", views.home)
]
```

When user visits:

/home/

Django calls the `home` view.

---

29. Explain Django Template Language (DTL).

Answer:

Django Template Language allows dynamic data display in HTML.

Features:

- Variables
- Tags
- Filters

Example:

Variable:

{{ name }}

Tag:

{% if user %}
Hello
{% endif %}

Filter:

{{ name|upper }}

---

30. Explain Template Inheritance in Django.

Answer:

Template inheritance allows one template to reuse another template's structure.

Example:

base.html:

<html>
<body>

```
{% block content %}
{% endblock %}

</body>
</html>
```

child.html:

```
{% extends 'base.html' %}

{% block content %}

<h1>Hello</h1>

{% endblock %}
```

Benefits:

- Code reuse
- Maintainable templates
- Less duplicate code

31. What are static files and media files in Django? Explain the difference.

Answer:

In Django, static files and media files are used to handle different types of files.

Static Files:

- Files created by developers.
- Used for website design and frontend.
- Includes CSS, JavaScript, images.

Example:

```
static/
   css/
   js/
   images/
```

Media Files:

- Files uploaded by users.
- Stored dynamically during application usage.

Examples:

- Profile pictures
- Documents
- Product images

Difference:

Static Files	Media Files
Developer-created files	User-uploaded files
CSS, JS, icons	Images, documents
Managed by developer	Generated by users

---

32. Explain Django Models and their importance.

Answer:

Django Model is a Python class that represents a database table.

Each model field represents a database column.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Importance:

- Defines database structure.
- Allows database operations using Python.
- Works with Django ORM.
- Reduces SQL writing.

---

33. Explain different Django Model Fields with examples.

Answer:

Django provides many built-in model fields.

CharField:

Stores short text.

Example:

name = models.CharField(max_length=100)

TextField:

Stores large text.

Example:

description = models.TextField()

IntegerField:

Stores integer values.

Example:

age = models.IntegerField()

BooleanField:

Stores True/False values.

Example:

active = models.BooleanField()

DateField:

Stores dates.

Example:

created = models.DateField()

EmailField:

Stores email addresses.

Example:

email = models.EmailField()

---

34. Explain Primary Key and Foreign Key in Django Models.

Answer:

Primary Key:

A primary key uniquely identifies each record in a database table.

Features:

- Unique value.
- Cannot be duplicated.
- Identifies a row.

Example:

id = models.AutoField(primary_key=True)

Foreign Key:

A Foreign Key creates a relationship between two models.

Example:

```
class Department(models.Model):
    name = models.CharField(max_length=50)


class Employee(models.Model):
    department = models.ForeignKey(
        Department,
        on_delete=models.CASCADE
    )
```

Here Employee is connected with Department.

---

35. Explain the difference between OneToOneField, ForeignKey, and ManyToManyField.

Answer:

OneToOneField:

Creates a one-to-one relationship.

Example:

One User → One Profile

profile = models.OneToOneField(User)

ForeignKey:

Creates a one-to-many relationship.

Example:

One Department → Many Employees

department = models.ForeignKey(Department)

ManyToManyField:

Creates many-to-many relationships.

Example:

Students and Courses

One student can have many courses.
One course can have many students.

courses = models.ManyToManyField(Course)

---

36. Explain Django Model Meta class.

Answer:

The Meta class is used to configure model options.

Example:

class Student(models.Model):

```
    name = models.CharField(max_length=100)

    class Meta:
        ordering = ['name']
        verbose_name = "Student"
```

Uses:

- Change table name.
- Set ordering.
- Add database options.
- Customize model behavior.

---

37. Explain the purpose of the `__str__()` method in Django Models.

Answer:

`__str__()` returns a human-readable string representation of a model object.

Example:

```
class Student(models.Model):

    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

Without `__str__()`:

Student object (1)

With `__str__()`:

Rahim

---

38. What are Django migrations? Explain makemigrations and migrate.

Answer:

Migrations are Django's way of applying database structure changes.

When we modify models, Django creates migration files.

Two main commands:

makemigrations:

Creates migration files.

Command:

python manage.py makemigrations

migrate:

Applies migrations to database.

Command:

python manage.py migrate

---

39. Explain Django ORM and its advantages.

Answer:

ORM (Object Relational Mapping) allows developers to interact with databases using Python objects instead of SQL queries.

Example:

SQL:

SELECT * FROM student;

Django ORM:

Student.objects.all()

Advantages:

- Less SQL code.
- Database independent.
- Faster development.
- Secure from SQL injection.
- Easy database management.

---

40. Explain the difference between get(), filter(), and exclude().

Answer:

get():

Returns a single object.

Example:

Student.objects.get(id=1)

If no object or multiple objects exist, it raises an error.

---

filter():

Returns multiple objects as QuerySet.

Example:

Student.objects.filter(age=20)

---

exclude():

Returns objects excluding given conditions.

Example:

Student.objects.exclude(age=20)

---

41. Explain QuerySet in Django with examples.

Answer:

QuerySet is a collection of database queries that retrieves objects from the database.

Example:

students = Student.objects.all()

Common QuerySet methods:

all()

filter()
exclude()
order_by()
values()

Advantages:

- Lazy loading.
- Efficient database operations.
- Chainable queries.

---

42. Explain the difference between select_related() and prefetch_related().

Answer:

Both are used for query optimization.

select_related():

- Uses SQL JOIN.
- Used for ForeignKey and OneToOne relationships.

Example:

Employee.objects.select_related('department')

---

prefetch_related():

- Uses separate queries.
- Used for ManyToMany and reverse relationships.

Example:

Student.objects.prefetch_related('courses')

Difference:

select_related	prefetch_related
Uses JOIN	Uses multiple queries
ForeignKey	ManyToMany

Faster for single relations	Better for multiple relations

---

43. Explain Django ORM aggregation functions.

Answer:

Aggregation functions perform calculations on database data.

Import:

```
from django.db.models import *
```

Count()

Counts records.

Example:

```
Student.objects.aggregate(
    Count('id')
)
```

Sum()

Adds values.

Example:

```
Order.objects.aggregate(
    Sum('price')
)
```

Avg()

Calculates average.

Example:

```
Student.objects.aggregate(
    Avg('marks')
)
```

Max()

Finds maximum value.

Min()

Finds minimum value.

---

44. Explain Django Q objects and F objects with examples.

Answer:

Q Object:

Used for complex queries using OR and AND conditions.

Example:

```
from django.db.models import Q

Student.objects.filter(
    Q(age=20) | Q(age=25)
)
```

F Object:

Used to compare model field values.

Example:

```
from django.db.models import F

Product.objects.filter(
    stock=F('sold')
)
```

---

45. Explain database relationships in Django ORM.

Answer:

Django supports three main relationships:

One-to-One:

One object relates to one object.

Example:

User and Profile.

One-to-Many:

One object relates to many objects.

Example:

Author and Books.

Implemented using:

ForeignKey

Many-to-Many:

Many objects relate to many objects.

Example:

Students and Courses.

Implemented using:

ManyToManyField

---

46. Explain Django Forms and their purpose.

Answer:

Django Forms simplify handling HTML forms and user input.

Functions:

- Generate HTML forms.
- Validate user input.
- Clean data.
- Handle submitted data.

Example:

```
from django import forms

class StudentForm(forms.Form):
    name = forms.CharField()
```

---

47. Explain the difference between Form and ModelForm.

Answer:

Form:

- Created manually.
- Fields are defined by developer.

Example:

```
class LoginForm(forms.Form):
    username = forms.CharField()
```

ModelForm:

- Automatically created from a Model.
- Reduces code.

Example:

```
class StudentForm(forms.ModelForm):

    class Meta:
        model = Student
        fields = '__all__'
```

---

48. Explain Django Form validation process.

Answer:

Form validation checks whether submitted data is correct.

Process:

1. User submits form.

2. Django receives data.
3. `is_valid()` method runs validation.
4. Cleaned data is created.
5. Errors are stored if validation fails.

Example:

```
if form.is_valid():
    print(form.cleaned_data)
```

---

49. Explain the use of clean() and clean_fieldname().

Answer:

clean():

Used for form-level validation.

Example:

```
def clean(self):
    data = super().clean()
    return data
```

---

clean_fieldname():

Used for specific field validation.

Example:

```
def clean_email(self):
    email = self.cleaned_data['email']

    if "@" not in email:
        raise forms.ValidationError(
            "Invalid email"
        )

    return email
```

---

50. Explain Django Form widgets and their uses.

Answer:

Widgets control how form fields are displayed in HTML.

Example:

```
name = forms.CharField(
   widget=forms.TextInput(
      attrs={
         'class':'form-control'
      }
   )
)
```

Common widgets:

- TextInput
- PasswordInput
- Textarea
- Select
- CheckboxInput
- FileInput

Uses:

- Customize HTML appearance.
- Add CSS classes.
- Change input types.

51. Explain Django REST Framework (DRF).

Answer:

Django REST Framework (DRF) is a powerful toolkit used to build Web APIs using Django.

It allows applications to communicate with other systems using JSON data.

Features:

- Serializer support
- Authentication system
- Permission handling
- API views
- Browsable API
- Pagination
- Validation

Example:

A mobile app can communicate with a Django backend using DRF API.

---

52. Explain the difference between Serializer and ModelSerializer.

Answer:

Serializer:

- Converts complex data types into JSON.
- Fields must be defined manually.

Example:

```
from rest_framework import serializers

class StudentSerializer(serializers.Serializer):
    name = serializers.CharField()
    age = serializers.IntegerField()
```

ModelSerializer:

- Automatically creates fields from Django models.
- Requires less code.

Example:

```
class StudentSerializer(serializers.ModelSerializer):

    class Meta:
        model = Student
        fields = '__all__'
```

Difference:

Serializer	ModelSerializer
Manual fields	Automatic fields
More control	Less code

Used for custom data	Used with models

---

53. Explain the process of converting Model data into JSON using Serializer.

Answer:

The process:

1. Retrieve model object.
2. Pass object to serializer.
3. Serializer converts data into Python native data.
4. Renderer converts it into JSON.

Example:

Model:

```
student = Student.objects.all()
```

Serializer:

```
serializer = StudentSerializer(student, many=True)
```

JSON output:

```
[
 {
  "name":"Rahim",
  "age":25
 }
]
```

---

54. Explain Serializer validation in Django REST Framework.

Answer:

Serializer validation checks whether incoming API data is correct.

Process:

1. Client sends data.

2. Serializer receives data.
3. `is_valid()` runs validation.
4. Errors are returned if data is invalid.

Example:

```
serializer = StudentSerializer(data=request.data)

if serializer.is_valid():
    serializer.save()
```

---

55. Explain the purpose of create() and update() methods in Serializer.

Answer:

create():

Used when creating a new object.

Example:

```
def create(self, validated_data):
    return Student.objects.create(**validated_data)
```

---

update():

Used when modifying an existing object.

Example:

```
def update(self, instance, validated_data):

    instance.name = validated_data['name']
    instance.save()

    return instance
```

---

56. Explain APIView in Django REST Framework.

Answer:

APIView is a DRF class used to create API endpoints.

It provides:

- HTTP method handling.
- Authentication support.
- Permission checking.
- Response handling.

Example:

```
from rest_framework.views import APIView
from rest_framework.response import Response


class StudentAPI(APIView):

    def get(self, request):
        return Response({
            "message":"Student List"
        })
```

---

57. Explain the difference between APIView and ViewSet.

Answer:

APIView:

- Uses separate methods for HTTP requests.
- More control.

Example:

```
get()
post()
put()
delete()
```

ViewSet:

- Groups related API actions together.
- Uses routers.

Example:

```
class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
```

Difference:

APIView	ViewSet
More control	Less code
Manual URL handling	Router based
Method based	Action based

---

58. Explain HTTP methods GET, POST, PUT, PATCH, DELETE.

Answer:

GET:

Used to retrieve data.

Example:

GET /students/

POST:

Used to create new data.

Example:

POST /students/

PUT:

Used for complete update.

Example:

PUT /students/1/

PATCH:

Used for partial update.

Example:

PATCH /students/1/

DELETE:

Used to remove data.

Example:

DELETE /students/1/

---

59. Explain REST API architecture.

Answer:

REST API is an architectural style for communication between client and server.

Components:

Client:
Requests data.

Server:
Processes request.

Resource:
Data objects.

HTTP Methods:

- GET
- POST
- PUT
- DELETE

Example:

Client
|
HTTP Request

|
Django REST API
|
Database
|
JSON Response

---

60. Explain authentication and authorization in Django.

Answer:

Authentication:

Checks who the user is.

Example:

- Login
- Username/password verification

Authorization:

Checks what the user can access.

Example:

- Admin permission
- User permission

Difference:

Authentication	Authorization
Identity checking	Permission checking
Login process	Access control

---

61. Explain Django built-in User authentication system.

Answer:

Django provides a built-in authentication framework.

Features:

- User model
- Password hashing
- Login/logout
- Permissions
- Groups

Import:

from django.contrib.auth.models import User

---

62. Explain login, logout, and authenticate functions in Django.

Answer:

authenticate():

Checks user credentials.

Example:

```
user = authenticate(
username="admin",
password="12345"
)
```

---

login():

Creates user session.

Example:

```
login(request,user)
```

---

logout():

Ends user session.

Example:

logout(request)

---

63. Explain Session and Cookie in Django.

Answer:

Session:

Stores user information on server side.

Example:

- Login status
- Shopping cart

Cookie:

Stores small data in browser.

Example:

- Remember preferences
- Session ID

Difference:

Session	Cookie
Server-side	Client-side
More secure	Less secure
Stores larger data	Small data

---

64. Explain CSRF attack and CSRF protection in Django.

Answer:

CSRF (Cross-Site Request Forgery) is an attack where a user performs unwanted actions without permission.

Example:

A hacker sends a fake money transfer request.

Django protection:

Uses CSRF Token.

Template:

{% csrf_token %}

It verifies that the request comes from the trusted website.

---

65. Explain Django Middleware and its uses.

Answer:

Middleware is a layer between request and response processing.

Flow:

Request
|
Middleware
|
View
|
Middleware
|
Response

Uses:

- Authentication
- Security
- Logging
- Session handling
- CSRF protection

---

66. Explain Django Signals.

Answer:

Signals allow different parts of Django applications to communicate when specific events happen.

Example:

After creating a user, automatically create a profile.

Common signals:

- pre_save
- post_save
- pre_delete
- post_delete

Example:

```
@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

---

67. Explain the difference between pre_save, post_save, pre_delete, and post_delete.

Answer:

pre_save:

Runs before saving an object.

Example:

- Modify data before save.

---

post_save:

Runs after saving an object.

Example:

- Create related profile.

---

pre_delete:

Runs before deleting an object.

---

post_delete:

Runs after deleting an object.

Difference:

Signal	Time
pre_save	Before save
post_save	After save
pre_delete	Before delete
post_delete	After delete

---

68. Explain Django Admin customization.

Answer:

Django Admin allows administrators to manage database data.

Customization includes:

- Changing display fields.
- Adding search.
- Adding filters.
- Custom actions.

Example:

```
@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):

    list_display = [
```

```
    'name',
    'age'
  ]

  search_fields = [
    'name'
  ]
```

---

69. Explain how to create a custom user model in Django.

Answer:

Custom user models allow developers to add extra user information.

Steps:

1. Create model by extending AbstractUser.

Example:

```
from django.contrib.auth.models import AbstractUser


class User(AbstractUser):

  phone = models.CharField(
    max_length=15
  )
```

2. Update settings:

```
AUTH_USER_MODEL = 'app.User'
```

Benefits:

- Flexible user management.
- Better project scalability.

---

70. Explain Django permissions and groups.

Answer:

Django provides permission-based access control.

Permissions:

Define what a user can do.

Examples:

- Add user
- Delete user
- Change data

Groups:

A collection of users with common permissions.

Example:

Admin Group
|
Permissions:
- Add product
- Delete product

Benefits:

- Better security.
- Easy user management.
- Role-based access control.

Part-5 (Django Advanced: CRUD API, JWT, Pagination, File Upload, Testing, Security)

Questions 71–90

71. Explain how to optimize database queries in Django.

Answer:

Database optimization improves application performance by reducing unnecessary database queries.

Techniques:

1. select_related():

Used for ForeignKey and OneToOne relationships.

Example:

Employee.objects.select_related('department')

It uses SQL JOIN.

---

2. prefetch_related():

Used for ManyToMany and reverse relationships.

Example:

Student.objects.prefetch_related('courses')

---

3. Using only() and defer():

Select only required fields.

Example:

Student.objects.only('name')

---

4. Database indexing:

Indexes make searching faster.

Example:

email = models.EmailField(db_index=True)

---

72. Explain Django caching and its benefits.

Answer:

Caching stores frequently used data temporarily to improve performance.

Types of Django cache:

1. Database Cache

Stores cache data in database.

2. File System Cache

Stores cache in files.

3. Memory Cache

Uses RAM for faster access.

4. Redis Cache

Commonly used in production.

Benefits:

- Faster response time.
- Reduces database load.
- Improves scalability.

Example:

```
from django.core.cache import cache

cache.set(
    'name',
    'Rahim',
    60
)
```

---

73. Explain how to create CRUD operations in Django.

Answer:

CRUD means:

- Create
- Read
- Update
- Delete

Create:

Insert new data.

Example:

```
Student.objects.create(
   name="Rahim"
)
```

Read:

Retrieve data.

Example:

```
Student.objects.all()
```

Update:

Modify existing data.

Example:

```
student.name="Karim"
student.save()
```

Delete:

Remove data.

Example:

```
student.delete()
```

---

74. Explain how to build a REST API using Django REST Framework.

Answer:

Steps to create API:

Step 1:

Install DRF:

```
pip install djangorestframework
```

Step 2:

Add app:

```
INSTALLED_APPS = [
'rest_framework'
]
```

Step 3:

Create Model:

```
class Product(models.Model):

    name=models.CharField(
        max_length=100
    )
```

Step 4:

Create Serializer:

```
class ProductSerializer(
    serializers.ModelSerializer
):

    class Meta:
        model=Product
        fields='__all__'
```

Step 5:

Create API View.

Step 6:

Add URL routing.

---

75. Explain the difference between Session Authentication and Token Authentication.

Answer:

Session Authentication:

- Uses Django session system.
- Common for web applications.
- Stores session information on server.

Example:

User login through browser.

Token Authentication:

- Uses a token for every request.
- Common for mobile applications and APIs.

Example:

Authorization:
Token abc123

Difference:

Session Authentication	Token Authentication
Server stores session	Client stores token
Browser based	API based
Uses cookies	Uses headers

---

76. Explain JWT Authentication in Django REST Framework.

Answer:

JWT (JSON Web Token) is a token-based authentication system.

JWT contains:

- Header
- Payload
- Signature

Flow:

User Login
|
Server Creates JWT Token
|
Client Stores Token
|
Client Sends Token
|
Server Verifies Token

Advantages:

- Stateless authentication.
- Suitable for mobile apps.
- Secure API communication.

---

77. Explain pagination in Django REST Framework.

Answer:

Pagination divides large API responses into smaller pages.

Example:

Instead of returning 10,000 records, API returns 10 records per page.

Benefits:

- Faster response.
- Less memory usage.
- Better user experience.

Example:

REST_FRAMEWORK = {

'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.PageNumberPagination',

'PAGE_SIZE':10

}

---

78. Explain API permissions in Django REST Framework.

Answer:

Permissions control who can access an API.

Common permissions:

AllowAny:

Anyone can access.

```
permission_classes=[
AllowAny
]
```

IsAuthenticated:

Only logged-in users.

```
permission_classes=[
IsAuthenticated
]
```

IsAdminUser:

Only admin users.

Benefits:

- Protect sensitive data.
- Control user access.

---

79. Explain how to handle file upload in Django.

Answer:

Django uses FileField for file uploading.

Model:

```
class Document(models.Model):

   file=models.FileField(
      upload_to='documents/'
   )
```

Form:

```
<form method="post"
enctype="multipart/form-data">
```

View:

```
request.FILES['file']
```

---

80. Explain how to handle image upload using ImageField.

Answer:

ImageField is used for image uploads.

Example:

```
class Profile(models.Model):

   image=models.ImageField(
      upload_to='profile/'
   )
```

Requirements:

Install Pillow:

```
pip install pillow
```

Settings:

```
MEDIA_URL='/media/'
MEDIA_ROOT='media/'
```

---

81. Explain Django testing framework.

Answer:

Django provides a built-in testing framework based on Python unittest.

Testing is used to verify:

- Models
- Views
- Forms
- APIs

Example:

```
from django.test import TestCase


class StudentTest(TestCase):

    def test_example(self):
        self.assertEqual(1,1)
```

Benefits:

- Finds bugs early.
- Improves code quality.
- Makes changes safer.

---

82. Explain Unit Testing in Django.

Answer:

Unit testing checks individual parts of an application.

Examples:

- Testing a function.
- Testing a model method.
- Testing a view.

Example:

```
def test_student_name(self):

    student=Student.objects.create(
```

```
        name="Rahim"
    )

    self.assertEqual(
        student.name,
        "Rahim"
    )
```

---

83. Explain how Django connects with databases.

Answer:

Django connects with databases using ORM.

Database configuration is written in:

settings.py

Example:

DATABASES = {

'default':{

'ENGINE':'django.db.backends.sqlite3',

'NAME':'db.sqlite3'

}

}

Supported databases:

- SQLite
- MySQL
- PostgreSQL
- Oracle

---

84. Explain the difference between SQLite, MySQL, and PostgreSQL in Django.

Answer:

SQLite:

- Default Django database.
- Lightweight.
- Good for development.

MySQL:

- Popular relational database.
- Good performance.
- Widely used.

PostgreSQL:

- Advanced open-source database.
- Supports complex queries.
- Recommended for large applications.

Difference:

SQLite	MySQL	PostgreSQL
Simple	Fast	Advanced
Development	Production	Enterprise

85. Explain Django security features.

Answer:

Django provides built-in security features:

CSRF Protection:

Prevents fake requests.

SQL Injection Protection:

ORM prevents unsafe SQL queries.

XSS Protection:

Escapes dangerous HTML.

Password Hashing:

Stores passwords securely.

Clickjacking Protection:

Uses security headers.

86. Explain SQL Injection and prevention methods in Django.

Answer:

SQL Injection is an attack where malicious SQL code is inserted into queries.

Example:

Unsafe:

query = "SELECT * FROM user WHERE id="+id

Django prevention:

Using ORM:

User.objects.filter(id=id)

Benefits:

- Parameterized queries.
- Safer database operations.

87. Explain XSS attack and prevention in Django.

Answer:

XSS (Cross-Site Scripting) allows attackers to inject malicious JavaScript.

Example:

<script>
alert('hack')
</script>

Django prevention:

- Automatic HTML escaping.
- Safe template system.
- Content security policies.

Example:

{{ username }}

is automatically escaped.

88. Explain the difference between null=True and blank=True.

Answer:

null=True:

Controls database value.

Allows NULL in database.

Example:

```
age=models.IntegerField(
null=True
)
```

blank=True:

Controls form validation.

Allows empty input.

Example:

```
name=models.CharField(
blank=True
)
```

Difference:

null=True	blank=True
Database level	Form level
Stores NULL	Allows empty input

89. Explain the difference between static files and uploaded media files.

Answer:

Static files:

- Created by developers.
- CSS, JS, images.
- Stored in static folder.

Media files:

- Uploaded by users.
- Stored in media folder.
- Example: profile images.

90. Explain Django project deployment requirements.

Answer:

Before deployment, Django requires:

1. Production Settings:

DEBUG=False

2. Web Server:

Examples:

- Gunicorn
- uWSGI

3. Reverse Proxy:

Example:

- Nginx

4. Database:

Production database like:

- PostgreSQL

5. Environment Variables:

Store:

- Secret key
- Database password
- API keys

6. Static File Configuration:

Run:

python manage.py collectstatic

Part-6 (Advanced Django + Error Handling)

Questions 91–100

91. Explain environment variables in Django.

Answer:

Environment variables are used to store sensitive configuration values outside the source code.

Examples:

- SECRET_KEY
- Database password
- API keys
- Email credentials

Example:

`.env` file:

```
SECRET_KEY=mysecretkey
DEBUG=False
DATABASE_PASSWORD=password123
```

Using environment variables improves:

- Security
- Configuration management
- Deployment process

92. Explain DEBUG mode in Django.

Answer:

DEBUG mode controls Django's error reporting behavior.

Development:

DEBUG = True

Features:

- Shows detailed error pages.
- Displays traceback.
- Helps developers debug.

Production:

DEBUG = False

Features:

- Hides sensitive information.
- Shows custom error pages.
- Improves security.

93. Explain the difference between development and production settings.

Answer:

Development Settings:

Used during application development.

Features:

- DEBUG=True
- SQLite database
- Detailed errors
- Local server

Production Settings:

Used for live applications.

Features:

- DEBUG=False
- Secure database
- Environment variables
- Optimized performance

Difference:

Development	Production
Testing purpose	Real users
Detailed errors	Hidden errors
Less security	High security

94. Explain Django management commands.

Answer:

Django management commands are command-line tools used to perform administrative tasks.

Examples:

Run server:

python manage.py runserver

Create migrations:

python manage.py makemigrations

Apply migrations:

python manage.py migrate

Create superuser:

python manage.py createsuperuser

95. Explain how to create custom Django commands.

Answer:

Custom commands allow developers to create their own management commands.

Structure:

```
app/
 |
 management/
   |
   commands/
     |
     mycommand.py
```

Example:

```
from django.core.management.base import BaseCommand


class Command(BaseCommand):

   def handle(self,*args,**kwargs):
       print("Custom command executed")
```

Run:

```
python manage.py mycommand
```

96. Explain Django middleware execution order.

Answer:

Middleware works between request and response.

Request flow:

```
User Request
     |
Middleware 1
     |
Middleware 2
     |
View
     |
Middleware 2
     |
Middleware 1
     |
Response
```

Important points:

- Request passes from top to bottom.
- Response passes from bottom to top.
- Middleware order affects application behavior.

Example middleware:

- SecurityMiddleware
- SessionMiddleware
- AuthenticationMiddleware
- CSRF Middleware

97. Explain Django signals with real-world examples.

Answer:

Signals allow applications to perform actions automatically when events happen.

Common signals:

- post_save
- pre_save
- post_delete

Real-world examples:

User Registration:

When a user is created:

- Automatically create profile.

Example:

```
@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):

    if created:
        Profile.objects.create(
            user=instance
        )
```

Other examples:

- Send email after order creation.
- Update inventory after purchase.
- Create audit logs.

98. Explain how Serializer handles nested relationships.

Answer:

Nested serializers allow displaying related model data inside another serializer.

Example:

Models:

```
class Author(models.Model):
    name=models.CharField(
        max_length=100
    )


class Book(models.Model):
    title=models.CharField(
        max_length=100
    )
    author=models.ForeignKey(
        Author,
        on_delete=models.CASCADE
    )
```

Serializer:

```
class AuthorSerializer(
    serializers.ModelSerializer
):

    class Meta:
        model=Author
        fields='__all__'


class BookSerializer(
    serializers.ModelSerializer
):

    author=AuthorSerializer()

    class Meta:
        model=Book
        fields='__all__'
```

Output:

```
{
"title":"Python",
"author":{
"name":"Rahim"
}
}
```

99. Explain how to create a complete CRUD API using Django REST Framework.

Answer:

CRUD API steps:

Step 1: Create Model

```
class Product(models.Model):

    name=models.CharField(
        max_length=100
    )

    price=models.IntegerField()
```

Step 2: Create Serializer

```
class ProductSerializer(
    serializers.ModelSerializer
):

    class Meta:
        model=Product
        fields='__all__'
```

Step 3: Create ViewSet

```
class ProductViewSet(
    viewsets.ModelViewSet
):

    queryset=Product.objects.all()

    serializer_class=ProductSerializer
```

Step 4: Add Router

```
router.register(
    'products',
    ProductViewSet
)
```

Available operations:

GET:

Retrieve data.

POST:

Create data.

PUT/PATCH:

Update data.

DELETE:

Remove data.

100. Explain complete Django application development workflow.

Answer:

A Django development workflow includes:

Step 1: Create Project

Command:

django-admin startproject projectname

Step 2: Create Application

Command:

python manage.py startapp appname

Step 3: Configure Settings

Add:

- Installed apps
- Database
- Middleware
- Static files

Step 4: Create Models

Define database structure.

Step 5: Create Migrations

Commands:

python manage.py makemigrations

python manage.py migrate

Step 6: Create Views

Implement business logic.

Step 7: Create Templates

Build user interface.

Step 8: Create APIs

Using Django REST Framework.

Step 9: Testing

Test:

- Models
- Views
- APIs

Step 10: Deployment

Configure:

- Production settings
- Database
- Web server
- Static files

- Security settings

Part-7 (Python Error Handling + Django Exception Handling)

Questions 101–120

101. Explain exception handling in Python.

Answer:

Exception handling is a technique used to handle runtime errors without stopping program execution.

Python uses:

- try
- except
- else
- finally

Example:

```
try:
    result = 10 / 2
    print(result)

except ZeroDivisionError:
    print("Cannot divide by zero")
```

Benefits:

- Prevents program crashes.
- Provides meaningful error messages.
- Improves application stability.

102. Explain try, except, else, and finally blocks in Python.

Answer:

try:

Contains code that may create an exception.

Example:

```
try:
    number = int(input())
```

except:

Handles the exception.

Example:

```
except ValueError:
    print("Invalid number")
```

else:

Executes when no exception occurs.

Example:

```
else:
    print("Success")
```

finally:

Always executes.

Example:

```
finally:
    print("Finished")
```

103. Explain different types of Python built-in exceptions.

Answer:

Common Python exceptions:

ValueError:

Occurs when value is incorrect.

Example:

```
int("hello")
```

TypeError:

Occurs when wrong data types are used.

Example:

5 + "10"

ZeroDivisionError:

Occurs when dividing by zero.

Example:

10/0

FileNotFoundError:

Occurs when a file does not exist.

Example:

open("test.txt")

KeyError:

Occurs when a dictionary key is missing.

Example:

data["age"]

104. Explain the difference between syntax error and runtime error.

Answer:

Syntax Error:

Occurs when Python code structure is incorrect.

Example:

```
if True
    print("Hello")
```

Runtime Error:

Occurs while executing the program.

Example:

10 / 0

Difference:

Syntax Error	Runtime Error
Before execution	During execution
Code writing mistake	Program execution problem

105. Explain the purpose of the raise keyword in Python.

Answer:

The `raise` keyword is used to manually generate an exception.

Example:

```
salary = -100

if salary < 0:
    raise ValueError(
        "Invalid salary"
    )
```

Uses:

- Custom validation.
- Creating meaningful errors.
- Controlling program flow.

106. Explain custom exceptions in Python.

Answer:

Custom exceptions are user-created exception classes.

Example:

```
class InvalidAgeError(Exception):
    pass
```

```
age = -5

if age < 0:
    raise InvalidAgeError(
        "Age is invalid"
    )
```

Benefits:

- Application-specific errors.
- Better error management.

107. Explain multiple exception handling in Python.

Answer:

Multiple exceptions can be handled using multiple except blocks.

Example:

```
try:

    number=int(input())
    result=10/number


except ValueError:

    print("Invalid input")


except ZeroDivisionError:

    print("Cannot divide by zero")
```

108. Explain the use of Exception class in Python.

Answer:

Exception is the base class for most Python errors.

Example:

```
try:
```

```
    x = 5 / 0

except Exception as e:

    print(e)
```

Advantages:

- Handles unexpected errors.
- Provides error details.

109. Explain file handling error handling in Python.

Answer:

File operations may generate different errors.

Example:

```
try:

    file=open(
        "data.txt",
        "r"
    )

    print(file.read())


except FileNotFoundError:

    print("File does not exist")


finally:

    print("Completed")
```

Common errors:

- FileNotFoundError
- PermissionError
- IOError

110. Explain Django error handling.

Answer:

Django provides different methods for handling errors.

Methods:

- try-except blocks
- Custom error pages
- Exception middleware
- Logging system

Common HTTP errors:

- 400 Bad Request
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

111. Explain Django 404 error handling.

Answer:

404 error occurs when a requested resource is not found.

Django automatically displays a 404 page.

Custom page:

Create:

```
templates/
   404.html
```

Example:

```
<h1>
Page Not Found
</h1>
```

112. Explain Django 500 server error handling.

Answer:

500 error occurs due to server-side problems.

Examples:

- Database errors.
- Programming mistakes.
- Unexpected exceptions.

Production setting:

DEBUG=False

Custom file:

templates/500.html

113. Explain exception handling inside Django views.

Answer:

Exceptions inside views can be handled using try-except.

Example:

```
def profile(request,id):

    try:

        user=User.objects.get(
            id=id
        )

    except User.DoesNotExist:

        return HttpResponse(
            "User not found"
        )
```

Advantages:

- Prevents crashes.
- Provides user-friendly messages.

114. Explain get_object_or_404() in Django.

Answer:

`get_object_or_404()` retrieves an object or returns a 404 response.

Example:

```
from django.shortcuts import get_object_or_404


student=get_object_or_404(
   Student,
   id=1
)
```

Advantages:

- Less code.
- Automatic error handling.

115. Explain Django model validation errors.

Answer:

Model validation checks whether model data is correct.

Example:

```
from django.core.exceptions import ValidationError


def validate_age(value):

   if value < 18:

      raise ValidationError(
         "Age must be 18 or above"
      )
```

Used for:

- Data validation.
- Business rules.

116. Explain Django Form error handling.

Answer:

Django forms automatically validate user input.

Example:

```
if form.is_valid():
```

```
    form.save()

else:

    print(form.errors)
```

Handles:

- Missing fields.
- Wrong format.
- Invalid values.

117. Explain Django REST Framework exception handling.

Answer:

DRF automatically handles API exceptions.

Examples:

- ValidationError
- AuthenticationFailed
- PermissionDenied
- NotFound

Example response:

```
{
"error":"Invalid data"
}
```

118. Explain custom exception handler in Django REST Framework.

Answer:

Custom exception handlers allow developers to modify API error responses.

Example:

```
from rest_framework.views import exception_handler


def custom_handler(exc, context):

    response = exception_handler(
```

```
        exc,
        context
    )

    return response
```

Benefits:

- Consistent API responses.
- Better error formatting.

119. Explain Serializer validation error handling in DRF.

Answer:

Serializer validates incoming API data.

Example:

```
serializer = StudentSerializer(
    data=request.data
)


if serializer.is_valid():

    serializer.save()

else:

    return Response(
        serializer.errors
    )
```

Handles:

- Required fields.
- Data type errors.
- Custom validation.

120. Explain why proper error handling is important in Django applications.

Answer:

Proper error handling improves application quality.

Benefits:

- Prevents application crashes.
- Improves security.
- Provides better user experience.
- Helps debugging.
- Makes applications easier to maintain.

Production applications should use:

- Logging
- Exception handling
- Monitoring
- Custom error pages